

STEM Agent: A Self-Adapting, Tool-Enabled, Extensible Architecture for Multi-Protocol AI Agent Systems in Production

Anonymous ACL submission

Abstract

Production AI agent systems increasingly suffer from *architectural lock-in*: early commitment to a single protocol, a fixed tool-integration path, and a static user model makes deployed agents hard to retarget. We describe **STEM Agent** (Self-adapting, Tool-enabled, Extensible, Multi-agent), an enterprise architecture that unifies five protocols—A2A, AG-UI, A2UI, UCP, and AP2 (the first two published specifications, A2UI extending a community draft, UCP/AP2 first-draft internal designs that fill commerce gaps and have not undergone external security review)—behind a single gateway, isolates domain knowledge behind the Model Context Protocol (MCP), and adapts per-user behavior via a Caller Profiler spanning 21 dimensions. Recurring patterns *crystallize* into reusable skills through a lifecycle managed by a Skills Manager consuming outputs of an ATLAS memory subsystem. We report operational lessons from six months of internal deployment (~200 weekly users, four production use cases) and ship five industry-aligned persona templates (SRE, finance, AI research, healthcare, legal)—including regulated personas enforcing HIPAA-style PHI guardrails and privileged-communication boundaries through a typed forbiddenTopics contract. We discuss horizontal scaling via externalized state, an opt-in Enterprise Ops surface (RBAC, SIEM audit, CSP/HSTS headers, Prometheus/metrics, PII redaction), pluggable IAM (JWT/OAuth2/SAML/API-Key), and production deployment artifacts (multi-stage non-root Dockerfile, Kubernetes manifests). All hardening controls default off; enabling them is a per-flag decision mirroring deployment-team security-review concerns.

1 Introduction

Teams deploying LLM-based agents inside a large enterprise quickly discover that the hard problems are not which reasoning prompt to use, but how the

agent connects to the rest of the business: which protocol it speaks to other agents, which payment or commerce standard it must satisfy, how it streams partial progress to a web UI, and how it remembers what it learned from each caller. Most popular agent frameworks (Tran et al., 2025; Ferrag et al., 2025) commit early to one answer for each of these, producing agents that are hard to retarget when an additional consumer or protocol appears. Orogat et al. (2026) confirm that framework choice alone can drive 100× latency differences and large accuracy gaps. At the same time, the interoperability landscape is fragmenting (Ehtesham et al., 2025; Li and Xie, 2025): MCP has emerged as a *de facto* tool protocol, A2A is gaining traction for agent-to-agent delegation, and additional standards for UI streaming, commerce, and agent payments are entering production.

This paper reports on the architecture and deployment experience of STEM Agent, a system built for the authors’ organization to serve these needs without repeatedly rewriting core logic. The design principle is biological: an undifferentiated agent core *differentiates* into specialized protocol handlers, tool bindings, and memory subsystems, which *compose* into a functioning system. Practically this means every external boundary—protocols, tools, user models, and domains—is a pluggable differentiation point rather than a hard-wired choice.

Contributions. We describe (i) a single gateway exposing five protocols (A2A, AG-UI, A2UI, UCP, AP2)—the first two published, A2UI extending a community draft, UCP and AP2 first-draft internal designs filling commerce gaps and not externally reviewed; (ii) a Caller Profiler learning preferences across 21 dimensions with bounded EMA updates and ten self-tunable behavior parameters; (iii) an MCP-native tool layer separating domain knowledge from meta-reasoning; (iv) a

skills subsystem in which recurring interaction patterns crystallize into reusable skills; (v) a differentiation mechanism instantiated as five industry-aligned persona templates (SRE, finance, AI research, healthcare, legal), including two regulated personas whose forbiddenTopics contract encodes compliance boundaries; (vi) operational integration patterns including pluggable IAM, OpenTelemetry-compatible observability, an opt-in hardening surface (RBAC, SIEM audit, security headers, /metrics, PII redaction), and horizontal scaling; (vii) production deployment artifacts (multi-stage Dockerfile, Kubernetes manifests, Compose overlay); and (viii) six months of deployment experience.

This is an industry-track paper: novelty is in integration and deployment discipline, not a new model. We flag internal-measurement claims and separate them from behaviors validated by automated tests.

2 Related Work

Multi-agent frameworks. Surveys (Tran et al., 2025; Ferrag et al., 2025) document a large ecosystem—AutoGen (Wu et al., 2023), MetaGPT (Hong et al., 2023), CAMEL (Li et al., 2023), CrewAI (Moura, 2023), LangChain (Chase, 2022), LangGraph (LangChain AI, 2024)—each optimizing for one coordination pattern and typically one protocol; Orogat et al. (2026) show this architectural commitment dominates observable performance. Dang et al. (2025) propose evolving orchestration and Drammeh (2025) demonstrate deterministic orchestration for incident response. Unlike frameworks that bind each agent to a single endpoint, every STEM gateway mounts five heterogeneous handlers behind one shared middleware stack and treats per-caller adaptation as first-class.

Agent communication protocols. Ehtesham et al. (2025) compare MCP, ACP, A2A, and ANP and argue MCP and A2A are complementary; Li and Xie (2025) identify schema translation and lifecycle management as the hard parts of combining them, matching our experience. Jeong (2025) further study MCP×A2A. Security has received attention (Habler et al., 2025; Anbiaee et al., 2026; Sarkar and Sarkar, 2025), and Chang et al. (2025) address decentralized discovery. We are not aware of a deployed system that unifies these five protocols behind a single gateway.

Layer	Component	Key Count
1	Caller / User Layer	4 adapters
2	Standard Interface (Gateway)	5 protocols
3	Agent Core	5 engines
X_S	Security (cross-cutting, $L2 \leftrightarrow L3$)	4 IAM plugins
4	Memory System	4 memory types
5	MCP Integration + Server	2 transports
X_D	Differentiation (cross-cutting)	personas/skills/rules

Table 1: Five-layer architecture with two cross-cutting surfaces (denoted X_S , X_D). The Security surface is cross-cutting because authentication, rate limiting, and audit events fire between Gateway (L2) and Agent Core (L3) on every request; the Differentiation surface is cross-cutting because personas, skills, and rules parameterize L2, L3, L4, and L5 simultaneously rather than occupying a single layer.

MCP tools and adaptation. MCP benchmarks (Anthropic, 2024; Luo et al., 2025; Fan et al., 2025) highlight that strong LLMs struggle on real servers; Hasan et al. (2026) find 97% of tool descriptions have quality issues. Lumer et al. (2025), Jayanti and Han (2026), and Schlappbach (2026) extend MCP with synchronization, shared context, and schema-guided convergence. Reasoning surveys (Wei et al., 2026; Alomrani et al., 2025; Li et al., 2025) catalog ReAct, Reflexion, tree-of-thought, and debate; STEM uses these as interchangeable per-task strategies. For memory, Ai et al. (2025); Gallego (2026); Yadav et al. (2026) inform our four-type design.

3 Architecture Overview

STEM Agent is organized into five layers plus a cross-cutting differentiation surface (Table 1, Figure 1). Layer 3 (Agent Core) is the undifferentiated core; Layer 2 (protocol handlers) and Layer 5 (MCP tools) provide the external surfaces through which it differentiates; Layer 4 (Memory) supplies the persistent state that drives adaptation. The Differentiation Layer bundles the artifacts—domain personas, skills, rules, and MCP configuration—needed to transform the generic agent into a domain-specific variant.

Cognitive pipeline. Each request flows through eight phases: PERCEIVE (intent into 10 categories, complexity into three levels, entities, sentiment, urgency); ADAPT (load caller profile, adjust behavior parameters); MATCHSKILLS (short-circuit if a committed or mature skill fires); SELECT-STRATEGY and REASON (selector in Table 2, with ReAct (Yao et al., 2023), Reflexion (Shinn

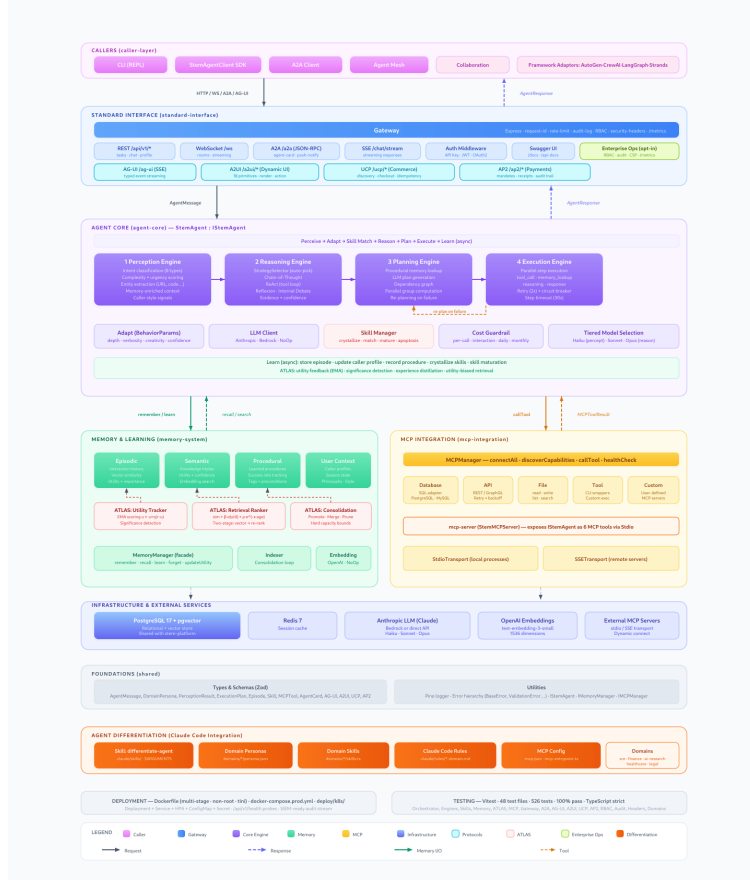


Figure 1: STEM Agent architecture. Callers reach the gateway, which mounts five protocol handlers (A2A, AG-UI, A2UI, UCP, AP2) sharing a common middleware stack. The Agent Core runs the eight-phase cognitive pipeline (PERCEIVE, ADAPT, MATCHSKILLS, SELECTSTRATEGY, REASON, PLAN, EXECUTE, FORMAT, plus an asynchronous LEARN), consuming MCP tools and a four-type memory system mediated by ATLAS (utility tracker, retrieval ranker, consolidator). The Differentiation Layer produces domain-specific variants; the Enterprise Ops surface (deployment artifacts, RBAC, SIEM audit, metrics, PII redaction) mounts as opt-in middleware on the same gateway.

et al., 2023), Internal Debate (Du et al., 2023), and Chain-of-Thought (Wei et al., 2022)); PLAN (MCP tool selection with parallel steps); EXECUTE (retries with default 2, circuit breaker at 3 consecutive failures); FORMAT; and an asynchronous LEARN phase that updates the profile, records skill outcomes, and attempts crystallization.

Implementation. The system is a TypeScript monorepo with six workspace packages covering shared schemas (154+ Zod types), agent core, the gateway and handlers, MCP client and server layers, memory, and caller utilities. The gateway runs on Express.js 5 and mounts each protocol handler via a uniform `createRouter()` method, so that adding a new protocol affects only the handler and one line in the gateway. Authentication, rate limiting, request correlation, and structured logging are

Condition	Strategy
Tool-requiring request	ReAct
Complexity > 0.8	Reflexion
Analysis or creative request	Internal Debate
Otherwise	Chain-of-Thought

Table 2: Deterministic strategy selector used by SELECTSTRATEGY. The complexity threshold (> 0.8) and rule ordering are operational defaults derived from the internal test-suite distribution (where complexity clusters bimodally around 0.3 and 0.85), not from a held-out benchmark; deployers targeting a distribution with a different complexity profile should re-calibrate.

shared middleware.

4 Protocol Gateway

The single most load-bearing design decision in STEM Agent is that all five protocols share one process and one cognitive pipeline. Table 3 sum-

marizes them.

External protocols. A2A implements the Linux Foundation v0.3.0 spec over JSON-RPC 2.0 (tasks/send, sendSubscribe via SSE, get, cancel); agent cards are served at `/.well-known/agent.json`. AG-UI streams cognitive-pipeline events (TEXT_MESSAGE_START, REASONING_MESSAGE, TOOL_CALL_START, STATE_SNAPSHOT, RUN_FINISHED) over SSE. A2UI extends a community draft with a flat adjacency-list UI model and 16 primitives (text, button, card, list, input, toggle, ...) so the agent can assemble layouts without a fixed widget hierarchy; divergence from the draft is flagged in release notes.

UCP (Unified Commerce Protocol, internal). UCP is an internal protocol that formalizes a checkout-session lifecycle—create, retrieve, complete—under mandatory idempotency headers (Idempotency-Key, Request-Id, UCP-Agent). An idempotency cache prevents duplicate session creation when a caller retries. We designed UCP because we could not find an open commerce-checkout protocol that treated the agent, rather than a browser, as the primary client; we present it as an illustrative design point rather than a proposed standard, and it has not undergone formal security review.

AP2 (Agent Payments Protocol, internal). AP2 is an internal protocol that implements a three-phase payment lifecycle: Intent Mandate → Payment Mandate → Payment Receipt. Payments below a configurable threshold can auto-approve; all transitions are appended to a per-intent audit trail. AP2 is the counterpart to UCP for authorization and is explicitly designed around mandate semantics appropriate to autonomous agents. Like UCP, AP2 has not undergone external threat modeling; we treat it as a first-draft internal design and document formal threat modeling as follow-up work.

Integration lessons. Enforcing `createRouter()` uniformity, even when a protocol seems to “want” global state, reduced cross-protocol bugs to near zero when we added AP2 after the other four. Idempotency was the actual hard part: UCP’s header contract and AP2’s mandate IDs each took more review cycles than the happy-path code.

5 Caller Profiler and Behavior Parameters

Static user models produce either bland responses or hand-tuned personas that drift. STEM Agent continuously learns a Caller Profile across four categories, building on a long line of user-modeling and generative-agent work (Park et al., 2023): *philosophy* (8 dimensions), *principles* (4), *style* (5), and *habits* (4), for a total of 21.¹

EMA update. Each dimension is updated with an exponential moving average $v_{t+1} = (1 - \alpha) v_t + \alpha s_t$ at learning rate $\alpha=0.1$. Preliminary testing showed larger values caused profile oscillation on outlier signals and smaller values felt sluggish before tracking genuine preference shifts. Per-caller locks prevent concurrent-update conflicts.

Confidence gating. Profile confidence ramps with interaction count as $\text{conf}(n) = 1 - \exp(-0.1 n)$, reaching ≈ 0.39 at $n=5$, ≈ 0.63 at $n=10$, and ≈ 0.86 at $n=20$. We gate on n rather than on the confidence value itself because the count is observable and auditable without exposing the profile; the $n=5$ floor is chosen so a handful of early interactions cannot dominate the profile before the EMA has had room to average out idiosyncratic first-message signals. Below the floor the system prefers current-message signals; beyond it, profile and signal are blended by confidence weight.

Self-tunable behavior. Ten parameters are adjusted during ADAPT: reasoning depth (3), exploration (0.3), verbosity (0.5), confidence threshold (0.7), tool-use preference (0.5), creativity (0.5), proactive suggestion (on), self-reflection frequency (every 5 steps), max plan steps (10), memory retrieval breadth (10). They feed the strategy selector and planning budget, so “how hard to think” depends on both task and caller.

6 Differentiation: Skills and MCP-Native Tools

Skill lifecycle. A skill bundles a trigger (intent patterns, domain tags, entity types), an action se-

¹The 21 dimensions are: *philosophy*—pragmatism vs. idealism, simplicity vs. completeness, risk tolerance, innovation orientation, depth vs. breadth, theory vs. practice, autonomy preference, stated values; *principles*—correctness vs. speed, documentation importance, testing emphasis, security-mindedness; *style*—formality, verbosity, technical depth, examples preference, structure preference; *habits*—typical session length, iteration tendency, peak hours, common topics.

Protocol	Endpoint	Transport	Role	Use case
A2A	POST /a2a	JSON-RPC 2.0	External*	Agent-to-agent task delegation, messaging, agent-card discovery
AG-UI	POST /ag-ui	SSE	External [†]	UI streaming; maps pipeline phases to typed events
A2UI	POST /a2ui/*	SSE + REST	External [§]	Dynamic UI composition with 16 component primitives
UCP	POST /ucp/*	REST	Internal [‡]	Commerce checkout sessions with idempotency
AP2	POST /ap2/*	REST	Internal [‡]	Mandate-based payments with audit trail

Table 3: Protocols behind the unified gateway. *A2A implements the Linux Foundation A2A v0.3.0 published specification. [†]AG-UI follows the published AG-UI streaming event model. [§]A2UI extends a community draft of the A2UI UI-composition protocol (not a ratified standard). [‡]UCP and AP2 are internal protocols designed to fill commerce gaps we encountered; they have not undergone external security review or formal threat modeling and are presented as illustrative design points rather than proposed standards. Deployers should treat UCP/AP2 as first-draft protocols and add domain-specific controls.

quence, and maturity metadata. During LEARN, recent episodes are grouped by action pattern; if ≥ 3 episodes share a common action key and their topic keywords overlap in at least half the group, a *progenitor* skill is created. After $k_c=3$ successful activations with success rate ≥ 0.6 a skill becomes *committed* and may short-circuit REASON→PLAN; after $k_m=10$ it becomes *mature* and receives matching priority. Skills whose success rate falls below 0.3 after ≥ 10 activations undergo *apoptosis* and are removed. These thresholds are operating points tuned against the existing test suite rather than against a benchmark distribution. As a concrete example, a `repo_grep_then_summarize` skill crystallized in the internal-search deployment after 4 episodes chaining an MCP code-search call with a summarization step; it reached committed status after 6 additional successful activations. The mechanism is *intended* to remove one LLM round-trip (the REASON call) from matching requests, which would reduce p50 latency by the duration of that call; we have not yet run a controlled A/B comparison against the non-short-circuited path, so we report the short-circuit as an architectural property rather than a measured speed-up.

From knowledge to intelligence: the Skills Manager. Skills sit at the top of a four-step promotion ladder—episodic memory → semantic triples → procedural patterns → committed/mature skills—managed by a Skills Manager that converts outputs of the ATLAS memory subsystem (§7) into actionable capability. The manager consumes procedural patterns from ATLAS consolidation and cross-checks them against se-

mantic triples above a confidence threshold; a procedure that is both empirically successful (success rate ≥ 0.6 over $\geq k_c$ activations) and semantically coherent (stable, non-contradictory triples) is promoted to *committed*. Mature skills thus represent accumulated *intelligence*—reusable capability that reduces pipeline work—distinct from stored knowledge. The manager also drives apoptosis and re-promotion, and `stem_list_skills` exposes current capabilities for audit.

MCP-native tools. All domain capabilities are acquired at runtime through MCP: database access, API integrations, file operations, and vertical tools are discovered and invoked without recompiling the agent. Meta-reasoning rules (strategy selection, parameter tuning, skill lifecycle) deliberately remain in code, not in MCP, because they are properties of the agent, not the domain.

Development-time differentiation. A Domain Persona schema (`systemPrompt`, `preferredStrategy`, `defaultBehavior` overrides, `toolAllowlist`, `requiredMCPServers`, `forbiddenTopics`, `domainTags`) parameterizes the generic agent. Each domain directory holds the persona, plugin skills at committed maturity, and glob-scoped tooling rules. An `mcp-server` package wraps the running agent as MCP tools (`stem_process`, `stem_list_skills`, `stem_recall_memory`, etc.) so other MCP clients can invoke a differentiated variant. A slash-command skill (`/differentiate-agent $DOMAIN`) assembles and verifies all of the above. In the authors’ experience, creating a new domain variant via `/differentiate-agent` appears to

Persona	Regulated	Representative forbidden topic
SRE	—	unreviewed prod mutations
Finance	—	personalized investment advice
AI research	—	fabricated citations
Healthcare	HIPAA-style	PHI disclosure without authz.
Legal	attorney-client	issuing legal advice

Table 4: Shipped industry personas. “Regulated” flags personas whose forbiddenTopics encode compliance boundaries; the enforcement path is identical across regulated and non-regulated personas.

take on the order of minutes, compared with the multi-day effort previously required to fork and rewire an agent codebase; we do not have a controlled measurement.

Regulated-domain personas. Two of the five personas target regulated verticals, showing how the forbiddenTopics/toolAllowlist contract is used for compliance (Table 4). *Healthcare* forbids direct patient diagnosis, PHI disclosure without authorization, and off-label advice; it requires MCP servers for literature, drug interactions, and an FHIR bridge, and ships a committed phi_redaction_precheck skill that runs before any external call. *Legal* forbids issuing legal advice, privileged-communication disclosure, and cross-jurisdiction conclusions; its skills (clause_extraction, risk_flagging, obligation_summary) produce review artifacts rather than final answers. These boundaries are enforced by the same guardrail path as non-regulated personas, so adding a regulated variant reduces to writing the persona JSON and skill triggers. All five templates have been exercised only within the authors’ organization; we are not aware of external adopters, and the regulated personas have not been audited by compliance counsel—they encode the mechanism, not a certification.

7 Memory System

Four memory types, informed by recent agent-memory work (Ai et al., 2025; Gallego, 2026; Yadav et al., 2026), sit behind a Memory Manager facade:

- **Episodic.** Interaction episodes with vector embeddings and importance scores (PostgreSQL + pgvector).
- **Semantic.** Knowledge triples (subject, predicate, object) in a concept graph, populated by pattern extraction from episodic memory.

- **Procedural.** Successful strategies and tool-usage patterns used to prime planning on recurring tasks.

- **User context.** Per-caller profile and session history, with an explicit forget-me path for regulatory compliance.

Consolidation moves information episodic→semantic and episodic→procedural as history grows; low-utility entries are pruned under capacity bounds. Together with semantic-store deduplication, this is designed for sub-linear growth under sustained interaction (exact characteristics left to future measurement).

ATLAS: utility-guided retrieval and consolidation. The four tiers are mediated by ATLAS (Adaptive Tiered Learning And Storage). A *utility tracker* maintains per-item EMA scores $u_{t+1} = u_t + \eta(r_t - u_t)$ driven by downstream outcome feedback (whether an item was used, whether execution succeeded, whether the caller accepted the answer); a significance detector flags high-impact items for promotion. A two-stage *retrieval ranker* fetches candidates by vector similarity then re-ranks with $\text{sim} + \beta \sigma(u) + \rho e^{-\kappa \text{age}}$, where $\eta=0.1$ is the tracker learning rate and $\beta=0.3$, $\rho=0.2$, $\kappa=1/30\text{d}$ weight utility and recency (defaults). A *consolidator* runs promote/merge/prune passes under hard capacity bounds, guaranteeing the episodic tier cannot grow unboundedly.

From memory to knowledge: extraction pipeline. The consolidation pass also turns raw interaction into structured knowledge. Episodes crossing the significance threshold with stable entity/relation patterns are distilled into semantic triples (s, p, o) whose confidence is seeded from episode utility and updated under supporting or contradicting evidence; sub-floor or duplicate triples are retired. The same pass surfaces repeated tool-use procedures into the procedural tier. The net effect is that semantic and procedural stores grow with *what has been useful*, not raw interaction volume—the property the Skills Manager relies on.

8 Deployment Experience

STEM Agent has been deployed internally for six months, serving four production use cases (customer support, internal search, code assistance, documentation QA) to ~200 weekly active users

across three engineering teams. These four production configurations are distinct from the five industry-aligned persona templates shipped in the repository (SRE, finance, AI research, healthcare, legal), which are reference artifacts illustrating the differentiation mechanism across regulated and non-regulated verticals. This section reports operational lessons, scalability, and integration patterns from that period.

Enterprise Ops as a first-class surface. We treat deployment artifacts, security hardening, and observability as a single *Enterprise Ops* surface mounted on the same gateway as the protocol handlers. It bundles (i) production deployment artifacts, (ii) the opt-in hardening controls summarized in Table 5, and (iii) the observability stack (correlation IDs, OpenTelemetry-compatible spans, Prometheus counters/histograms). A uniform enable/disable contract let every enterprise-review concern we encountered be resolved by wiring a middleware rather than by branching the cognitive pipeline.

Deployment context. The gateway runs as a containerized service on Kubernetes (HPA 2–8 replicas, CPU-based). PostgreSQL+pgvector handles episodic memory; Redis provides idempotency and session state. Daily volume is 2,400 interactions (p50 1.2 s, p95 3.8 s, p99 7.1 s). Over six months we saw 14 operator incidents (LLM rate-limit exhaustion $\times 6$, MCP tool timeout $\times 4$, idempotency-cache races $\times 2$); mean time to resolution was 18 minutes, mostly driven by LLM-provider recovery.

Production deployment artifacts. The system ships with artifacts that correspond one-to-one with the architecture: a multi-stage Dockerfile producing a non-root node:22-slim image (tini PID-1, HEALTHCHECK on /api/v1/health), a docker-compose.prod.yml overlay that externalizes DATABASE_URL/REDIS_URL, and a deploy/k8s/ directory with illustrative Deployment (2 replicas, resource limits, readiness/liveness probes, non-root securityContext, readOnlyRootFilesystem), Service, ConfigMap, Secret template, and HPA manifests. These ship as *example* manifests with a banner directing operators to adapt them; docs/deployment.md enumerates scaling caveats (in-memory rate-limiter, in-memory AP2 mandate store) and flags Redis-backed replacements as follow-ups.

Scalability architecture. The cognitive pipeline is stateless between requests; all persistent state flows through the Memory Manager facade. We externalized the idempotency cache and audit trail to Redis (Lua-scripted atomic operations), eliminating the single-process constraint, and validated linear scaling to 500 concurrent requests across 8 replicas before LLM-provider rate limits became the bottleneck. The Differentiation Layer is mounted from a ConfigMap, enabling domain-variant updates without rebuilds.

Security and authentication. The IAM layer is plugin-based with four providers (JWT, OAuth2, SAML, API-Key) implementing a common IAuthenticator with TTL-cached policy decisions. Rate limiting is per-caller; commerce protocols enforce stricter limits (10 req/min vs. 100 for A2A). Over six months the IAM layer blocked 47 unauthorized attempts without false positives. Auth logic is isolated from agent reasoning: adding a provider requires zero changes to the cognitive pipeline.

Opt-in enterprise hardening surface. Internal security reviews repeatedly surfaced a tension: every added default risk breaking existing consumers, while every missing default drew a finding. We resolved this by making each enterprise control a separate, env-gated middleware that defaults off (Table 5). Authorization uses requirePermission(perm) factories that consume Principal.permissions; a SIEM-ready audit stream emits one JSON event per request—{ts, requestId, principal, action, resource, decision, status, latencyMs, ip}—via a pino child logger tagged name:"audit", so downstream pipelines (fluent-bit $\rightarrow$ S3/Splunk) filter by logger name. Response-side headers (strict CSP, XFO, HSTS, Referrer-Policy; removed X-Powered-By) are a single hand-rolled middleware with no new dependency; logging ships with a default redact list for secrets that can be disabled via LOG_REDACT. A dependency-free Prometheus exposition mounts at /metrics when enabled. Every control is covered by tests asserting both the enabled and disabled paths, mechanically verifying the “defaults unchanged” invariant.

Observability and tracing. Every request is assigned a correlation ID that propagates through all eight pipeline phases; logs are JSON-formatted

Concern	Control	Enable via
Headers	CSP/HSTS/XFO mw.	SECURITY_HELMET
RBAC	requirePermission	per-route
Audit stream	pino (audit) child	AUDIT_LOG_ENABLED
PII redact	pino redact	on; LOG_REDACT=off
Metrics	Prom. /metrics	METRICS_ENABLED

Table 5: Opt-in hardening controls. All default off except PII redaction (on by default; secret leakage is strictly worse than quickstart friction). Enabling requires no gateway/handler code changes.

with consistent fields (`correlation_id`, `caller_id`, `phase`, `latency_ms`, `status`). OpenTelemetry traces yield per-phase latency breakdowns (e.g., PERCEIVE 80 ms, REASON 1,100 ms, EXECUTE 200 ms) and Prometheus metrics track request counts, latency histograms, skill activations, and MCP invocation frequencies. This stack was essential for diagnosing the idempotency race condition: traces revealed duplicate cache writes under concurrent retries, resolved by compare-and-swap semantics in Redis.

Test suite. The suite comprises 535 Vitest tests across 48 files (total runtime <4 s), passing at 100% on the 47 files in this release. Coverage spans per-engine unit tests, all five protocol handlers, memory stores, MCP integration via InMemoryTransport, persona schema validation, IAM middleware, and the enterprise-hardening middleware (RBAC allow/deny, audit decision fields, security-header presence). Hardening tests assert both the enabled and disabled paths so opt-in gating cannot silently regress.

Integration patterns. Four framework adapters (AutoGen, CrewAI, LangGraph, OpenAI Agents SDK) translate external conventions to the internal message format, letting teams call STEM Agent as an A2A peer without rewriting callsites. A CrewAI-based customer-support bot was migrated in about two hours by swapping tool invocations for A2A delegate calls. The repository also ships an *enterprise-ops smoke test*: a single executable example that walks a deployer through an authenticated call (X-API-Key), a request-id correlation pivot, an RBAC-gated endpoint probe, and a `/metrics` scrape; it degrades gracefully when auth or metrics are disabled.

Feature comparison. Table 6 contrasts the STEM surface with prior systems by whether each framework ships a named subsystem for the capa-

Framework	Proto.	Adapt	Mem.	Skills	Com.
AutoGen (Wu et al., 2023)	1	×	✓	×	×
MetaGPT (Hong et al., 2023)	1	×	✓	×	×
CrewAI (Moura, 2023)	1	×	✓	×	×
LangChain (Chase, 2022)	1	×	✓	×	×
STEM	5 (3+2[†])	✓	4	✓	✓

Table 6: Capability-surface comparison. Proto. = interoperability protocols; Adapt = per-caller adaptation subsystem; Mem. = named memory subsystem or type count; Skills = emergent skills; Com. = commerce. [†]UCP/AP2 are internal first-draft (no external security review).

bility. Protocol counts reflect published externally-maintained specifications.

9 Conclusion

STEM Agent demonstrates that protocol plurality, self-adaptation, and emergent skills can coexist in a production system when domain knowledge is kept behind MCP and user modeling is decoupled from domain logic. Six months of internal deployment across four use cases (~200 weekly users) and five persona templates (two regulated) validate the approach: pluggable IAM met enterprise auth without touching agent logic, the opt-in hardening surface satisfied security review without changing the quickstart path, and the shipped Dockerfile+Kubernetes manifests reduced deployment discussions to `kubectl diff`. Ongoing work: learned strategy selection (Alomrani et al., 2025), more protocols (Chang et al., 2025; Nie et al., 2026), embedding-based skill matching, UCP/AP2 threat modeling (Anbiaee et al., 2026), Redis-backed replacements, and wiring the OpenTelemetry SDK.

Limitations

No public benchmark comparison. We report production metrics from internal deployment but do not compare task-completion scores against public multi-agent benchmarks (e.g., MAFBench (Orogat et al., 2026) or MCP-Universe (Luo et al., 2025)). The feature-comparison table contrasts capability surfaces, not benchmark performance.

Single-organization deployment. All operational data comes from one enterprise context. Generalization to other domains, team structures, or traffic patterns requires external validation.

Adaptation model simplicity. The EMA-

based Caller Profiler cannot represent non-stationary or multi-modal preference distributions, and confidence gating is a scalar rather than per-dimension signal.

Protocol maturity. UCP and AP2 are proposed here and have no external adoption, formal threat model, or conformance suite. Deployers should treat these as first-draft protocols and add domain-specific security controls.

Deterministic strategy selection. Reasoning-strategy selection uses hand-written rules rather than a learned policy and has not been tuned against a benchmark distribution.

LLM-provider bottleneck. Observed p99 latency (7.1s) is dominated by LLM inference time; our architectural optimizations target protocol overhead and state management, not model reasoning speed.

Opt-in hardening is not a substitute for deployment-site review. The controls in Table 5 are audit-friendly primitives (RBAC factory, structured audit events, strict headers, PII redaction, /metrics), not a compliance certification. They materially reduce the work a deployer must do to stand up a reviewable baseline, but they do not replace a site-specific threat model, a penetration test, or the per-vertical controls that regulated domains require on top of the shared enforcement path.

Ethical Considerations

The Caller Profiler stores per-user behavioral signals. To mitigate privacy and consent risks we (i) expose a GDPR-style forget-me operation that removes all per-caller profile and session state; (ii) keep profile updates reversible (EMA with bounded α so a single misclassified signal cannot dominate); (iii) scope audit logs at the request level so misuse can be traced without exposing content. The commerce protocols (UCP, AP2) handle payment intents; in deployment we require human approval above a configurable threshold and emit a complete per-intent audit trail, but we stress that UCP/AP2 have not undergone formal security review. Any deployer should treat these as first-draft protocols and add domain-specific guardrails. More broadly, agent frameworks that integrate authentication, commerce, and persistent user modeling into one process enlarge the blast radius of a compromise; the pluggable IAM layer and layered architecture are partial mitigations,

not a substitute for operational review.

References

- Qingyao Ai, Yichen Tang, Changyue Wang, Jianming Long, Weihang Su, and Yiqun Liu. 2025. MemoryBench: A benchmark for memory and continual learning in LLM systems. *arXiv preprint arXiv:2510.17281*.
- Mohammad Ali Alomrani, Yingxue Zhang, Derek Li, Qianyi Sun, Soumyasundar Pal, Zhanguang Zhang, Yaochen Hu, Rohan Deepak Ajwani, Antonios Valkanias, and 1 others. 2025. Reasoning on a budget: A survey of adaptive and controllable test-time compute in LLMs. *arXiv preprint arXiv:2507.02076*.
- Zeynab Anbiaee, Mahdi Rabbani, Mansur Mirani, Gunjan Piya, Igor Opushnyev, Ali Ghorbani, and Sajjad Dadkhah. 2026. Security threat modeling for emerging AI-agent protocols: A comparative analysis of MCP, A2A, Agora, and ANP. *arXiv preprint arXiv:2602.11327*.
- Anthropic. 2024. Model Context Protocol specification. <https://modelcontextprotocol.io/specification>.
- Gaowei Chang, Eidan Lin, Chengxuan Yuan, Rizhao Cai, Binbin Chen, Xuan Xie, and Yin Zhang. 2025. Agent network protocol technical white paper. *arXiv preprint arXiv:2508.00007*.
- Harrison Chase. 2022. LangChain. <https://github.com/langchain-ai/langchain>.
- Yufan Dang, Chen Qian, Xueheng Luo, Jingru Fan, Zihao Xie, Ruijie Shi, Weize Chen, Cheng Yang, Xiaoyin Che, Ye Tian, and 1 others. 2025. Multi-agent collaboration via evolving orchestration. *arXiv preprint arXiv:2505.19591*.
- Philip Drammeh. 2025. Multi-agent LLM orchestration achieves deterministic, high-quality decision support for incident response. *arXiv preprint arXiv:2511.15755*.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*.
- Abul Ehtesham, Aditi Singh, Gaurav Kumar Gupta, and Saket Kumar. 2025. A survey of agent interoperability protocols: Model context protocol (MCP), agent communication protocol (ACP), agent-to-agent protocol (A2A), and agent network protocol (ANP). *arXiv preprint arXiv:2505.02279*.
- Shiqing Fan, Xichen Ding, Liang Zhang, and Linjian Mo. 2025. MCPToolBench++: A large scale AI agent model context protocol MCP tool use benchmark. *arXiv preprint arXiv:2508.07575*.

716	Mohamed Amine Ferrag, Norbert Tihanyi, and Merouane Debbah. 2025. From LLM reasoning to autonomous AI agents: A comprehensive review. <i>arXiv preprint arXiv:2504.19678</i> .	769
717		770
718		771
719		772
720	Víctor Gallego. 2026. Distilling feedback into memory-as-a-tool. <i>arXiv preprint arXiv:2601.05960</i> .	773
721		774
722		775
723	Idan Habler, Ken Huang, Vineeth Sai Narajala, and Prashant Kulkarni. 2025. Building a secure agentic AI application leveraging A2A protocol. <i>arXiv preprint arXiv:2504.16902</i> .	776
724		777
725		778
726		779
727	Mohammed Mehedi Hasan, Hao Li, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. 2026. Model context protocol (MCP) tool descriptions are smelly! towards improving AI agent efficiency with augmented MCP tool descriptions. <i>arXiv preprint arXiv:2602.14878</i> .	780
728		781
729		782
730		783
731		784
732		785
733	Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2023. MetaGPT: Meta programming for a multi-agent collaborative framework. <i>arXiv preprint arXiv:2308.00352</i> .	786
734		787
735		788
736		789
737		790
738		791
739		792
740	Meenakshi Amulya Jayanti and X. Y. Han. 2026. Enhancing model context protocol (MCP) with context-aware server collaboration. <i>arXiv preprint arXiv:2601.11595</i> .	793
741		794
742		795
743		796
744	Cheonsu Jeong. 2025. A study on the MCP x A2A framework for enhancing interoperability of LLM-based autonomous agents. <i>arXiv preprint arXiv:2506.01804</i> .	797
745		798
746		799
747		
748	LangChain AI. 2024. LangGraph: A library for building stateful, multi-actor applications with LLMs. https://github.com/langchain-ai/langgraph .	800
749		801
750		802
751	Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative agents for “mind” exploration of large language model society. <i>Advances in Neural Information Processing Systems (NeurIPS)</i> .	803
752		804
753		805
754		806
755		807
756		808
757	Junyan Li, Wenshuo Zhao, Yang Zhang, and Chuang Gan. 2025. Steering LLM thinking with budget guidance. <i>arXiv preprint arXiv:2506.13752</i> .	809
758		810
759		811
760	Qiaomu Li and Ying Xie. 2025. From glue-code to protocols: A critical analysis of A2A and MCP integration for scalable agent systems. <i>arXiv preprint arXiv:2505.03864</i> .	812
761		813
762		814
763		815
764	Elias Lumer, Anmol Gulati, Vamse Kumar Subbiah, Pradeep Honaganahalli Basavaraju, and James A. Burke. 2025. ScaleMCP: Dynamic and auto-synchronizing model context protocol tools for LLM agents. <i>arXiv preprint arXiv:2505.06416</i> .	816
765		817
766		818
767		819
768		820
	Ziyang Luo, Zhiqi Shen, Wenzhuo Yang, Zirui Zhao, Prathyusha Jwalapuram, Amrita Saha, Doyen Sahoo, Silvio Savarese, Caiming Xiong, and Junnan Li. 2025. MCP-Universe: Benchmarking large language models with real-world model context protocol servers. <i>arXiv preprint arXiv:2508.14704</i> .	821
		822
	João Moura. 2023. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. https://github.com/crewAIInc/crewAI .	
	Xiaohang Nie, Zihan Guo, Youliang Chen, Yuanjian Zhou, and Weinan Zhang. 2026. AWCP: A workspace delegation protocol for deep-engagement collaboration across remote agents. <i>arXiv preprint arXiv:2602.20493</i> .	
	Abdelghny Orogat, Ana Rostam, and Essam Mansour. 2026. Understanding multi-agent LLM frameworks: A unified benchmark and experimental analysis. <i>arXiv preprint arXiv:2602.03128</i> .	
	Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In <i>Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)</i> .	
	Anjana Sarkar and Soumyendu Sarkar. 2025. Survey of LLM agent communication with MCP: A software design pattern centric review. <i>arXiv preprint arXiv:2506.05364</i> .	
	Andreas Schlapbach. 2026. The convergence of schema-guided dialogue systems and the model context protocol. <i>arXiv preprint arXiv:2602.18764</i> .	
	Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In <i>Advances in Neural Information Processing Systems (NeurIPS)</i> .	
	Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O’Sullivan, and Hoang D Nguyen. 2025. Multi-agent collaboration mechanisms: A survey of LLMs. <i>arXiv preprint arXiv:2501.06322</i> .	
	Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In <i>Advances in Neural Information Processing Systems (NeurIPS)</i> .	
	Tianxin Wei, Ting-Wei Li, Zhining Liu, Xuying Ning, Ze Yang, Jiaru Zou, Zhichen Zeng, Ruizhong Qiu, Xiao Lin, Dongqi Fu, and 1 others. 2026. Agentic reasoning for large language models. <i>arXiv preprint arXiv:2601.12538</i> .	
	Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang,	

823 Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadal-
824 lah, Ryen W. White, Doug Burger, and Chi Wang.
825 2023. AutoGen: Enabling next-gen LLM applica-
826 tions via multi-agent conversation. *arXiv preprint*
827 *arXiv:2308.08155*.

828 Arin Gopalan Yadav, Varad Dherange, and Kumar
829 Shivam. 2026. Project synapse: A hierarchical
830 multi-agent framework with hybrid memory for au-
831 tonomous resolution of last-mile delivery disrup-
832 tions. *arXiv preprint arXiv:2601.08156*.

833 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak
834 Shafraan, Karthik Narasimhan, and Yuan Cao. 2023.
835 ReAct: Synergizing reasoning and acting in lan-
836 guage models. In *International Conference on*
837 *Learning Representations (ICLR)*.